

User Requirement & Specification Report of Campus Card Service System

李荣耀 2024103975 (Leader), 陈抟飞 2024101329, 杨启承 2024101322

First Version, May 2026

Table of Contents

1 Business Logic Investigation	2
1.1 Investigation Method	2
1.2 Key Finding: Peculiarity of the Top-Up Process	4
1.3 Investigation Conclusions	5
2 Problem Description	6
2.1 Background	6
2.2 Existing Pain Points	6
2.3 Project Objective	6
2.4 Terminology Definition	7
3 Proposed Solution	7
3.1 Solution Overview	7
3.2 Core Business Logic, Redesigned	8
3.3 System Boundary and Responsibility Definition	8
4 User Requirement Analysis	9
4.1 System Requirements	9
4.2 Acceptance Tests	11
5 User Requirement Specification	12
5.1 Use Cases	12
5.2 Acceptance Tests	22
6 Top UI Design	28
7 System Specification	28
7.1 System Responsibility Boundaries	28
7.2 Functional Modules	29
7.3 System Architecture Design	29

7.4 Core Interface Operational Flows 31

7.5 Deployment and Interaction Highlights 32

1 Business Logic Investigation

1.1 Investigation Method

To accurately capture the real business logic and potential pain points of campus card services, our project team adopted a “**process immersion method**” to conduct an end-to-end hands-on experience and in-depth analysis of the current campus card top-up process at Jinan University.

The investigation found that the campus card top-up entry is integrated into the “Entity Campus Card Service” module under the “Micro Services” section of the official “Jinan University” WeChat public account. As shown in figure 1, the service supports two types of wallets: “Card Account” and “Venue Electronic Account.” Currently, the only available payment channels are WeChat Pay and bank card payments linked to WeChat.

The payee displayed during payment is the full name of Jinan University, i.e., the university’s corporate basic account.

×

卡片充值

weixinfwh.jnu.edu.cn

...

姓 名

李荣耀

账 号

余 额

7.00元

钱包类型

卡账户

 卡账户



 场馆电子帐户

支付方式

微信

 微信



 绑定银行卡

金 额

请输入金额

30元

50元

100元

200元

300元

500元

服 务 时 间：01:00-22:55

确认充值

温馨提示：校园卡充值，充值金额首先进入过渡余额，需要

Figure 1: Screenshot of recharge page in JNU campus card service web client

1.2 Key Finding: Peculiarity of the Top-Up Process

During the investigation, we identified a critical mechanism that deviates from common online payment perceptions—the “Interim Balance” mechanism, as shown in figure 2.

The complete process is as follows:

- **Payment completion does not equal balance increase:** After the user successfully transfers funds to the university’s corporate account via WeChat Pay, the amount is not directly added to the campus card’s usable balance.
- **Funds temporarily held:** The top-up amount first enters an intermediate state called “Interim Balance.”
- **Offline activation:** The system prompts the user to swipe their card at any campus canteen POS terminal. Upon successful swiping, the “Interim Balance” is officially transferred to the “Card Balance,” which can then be used for consumption.



Figure 2: Screenshot of interim balance page in JNU campus card service web client

1.3 Investigation Conclusions

- **Fund flow:** User funds → WeChat Pay → University corporate basic account → Campus card system receives asynchronous notification of successful WeChat Pay → System writes “Interim Balance” to the user’s campus card account → User swipes card at campus POS terminal → System transfers “Interim Balance” to “Card Balance.”

- **Information flow:** WeChat Pay platform generates a successful payment message → Sends the message (including order ID, amount, timestamp, etc.) to the campus card system via an API callback interface → Campus card system validates the message → System pushes the “top-up success” status and “pending amount” to the frontend → During card swiping at the POS terminal, the card chip interacts with the backend system, initiating a load command → System validates and updates the balance field in the database, recording the transaction.
- **Key fact:** The campus card system acts purely as a “ledger” throughout the entire fund flow process. It is responsible for recording and updating user entitlements (balances) but never actually holds, manages, or operates the university’s physical bank funds.

2 Problem Description

2.1 Background

According to our investigation, the campus card penetration rate at Jinan University is nearly 100%. It serves as an indispensable infrastructure for identity verification and on-campus consumption, covering key scenarios such as canteen dining, supermarket shopping, and library access. However, high adoption has not translated into high usage. The underlying reason lies in significant user experience interruptions in the current online service process.

2.2 Existing Pain Points

- **Single payment channel:** Currently, only WeChat Pay is supported, excluding Alipay, which has a vast user base, thereby limiting users’ payment choices.
- **Offline interruption in top-up process:** The most prominent pain point is the “Interim Balance” mechanism. After users conveniently complete payment on mobile devices, they are forced to physically go to a POS terminal to “swipe and load.” This process drags an otherwise efficient online service back offline, greatly undermining the core value of the mobile application and causing a fragmented user experience.
- **Lack of real-time experience:** The temporal and spatial delay between successful online payment and offline card usability fails to meet users’ expectations of “instant availability” upon successful payment.

2.3 Project Objective

Based on the above pain points, this project aims to design and describe a new-generation “Campus Card Service System (CCSS).” The core objective is to eliminate the outdated offline confirmation step represented by the “Interim Balance” and **reshape a purely online, multi-channel, real-time closed-loop experience**, elevating campus card service from merely “usable” to “delightful,” thereby significantly enhancing teachers’ and students’ willingness and stickiness.

2.4 Terminology Definition

To ensure precision and unambiguity in this document, the core terms used in this project are defined in the table below. These definitions also reference the actual business practices observed at Jinan University.

- **Campus Card** A medium uniformly issued by the university that integrates identity authentication and electronic payment functions. It can be a physical card or a virtual card within an app. The monetary value associated with its account can only be used for consumption in designated on-campus scenarios.
- **Card Balance** The monetary value recorded in the campus card backend account, representing the amount a user can directly spend. Investigation shows this balance is non-withdrawable and non-refundable.
- **Interim Balance** A temporary fund state specific to the current process. After a user successfully completes an online payment, the funds first enter this state and must be loaded via swiping at a POS terminal before being converted to “Card Balance.” This project aims to eliminate this state.
- **Corporate Account** The legal entity account opened by the university at a bank, used to collect all user campus card top-up funds. Our system has no operational authority over this account and only receives proof of fund arrival via bank or third-party payment platform interfaces.
- **Top-up** The complete service process whereby a user initiates a transaction through our system, completes the fund transfer via an external payment platform, and ultimately increases the campus card account balance.
- **Payment Callback** An asynchronous HTTP request initiated by a third-party payment platform to its integrated service provider system after a user successfully completes a payment. This request is used to notify the service provider that a payment transaction has been officially confirmed.

3 Proposed Solution

3.1 Solution Overview

To completely resolve the offline interruption in the current process, we propose designing a mobile web application based on a B/S architecture—the CCSS. The core philosophy of this system is “Pay & Instant Credit.” We advocate that once a user completes payment via WeChat Pay or Alipay, the entire user-system interaction loop should be finalized online, without any additional offline actions.

3.2 Core Business Logic, Redesigned

The key transformation introduced by this solution is the elimination of the “Interim Balance” state. The new business process is redesigned as follows:

1. **User Initiation:** The user selects the top-up amount and payment method (WeChat Pay / Alipay) within the CCSS App.
2. **Platform Payment:** The system calls the respective third-party payment SDK; the user completes identity verification and payment; funds are transferred to the university’s corporate account.
3. **Callback Confirmation:** The system backend receives the successful payment callback message from WeChat Pay or Alipay and validates its legitimacy.
4. **Direct Balance Credit:** Upon successful validation, the system no longer creates an “Interim Balance.” Instead, it directly and atomically increases the user’s campus card account usable balance.
5. **Real-time Feedback:** The frontend page updates the balance in real time and pushes a notification to the user: “Top-up successful, XX yuan has been credited.”

The comparison of old and new processes are visualized as figure 3 and 4.

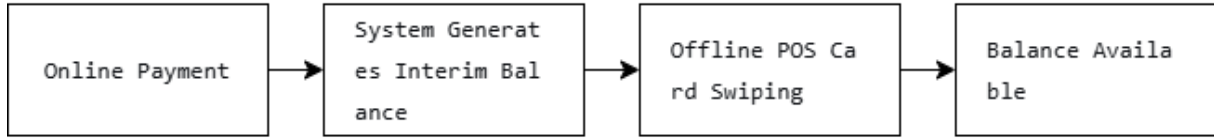


Figure 3: Diagram of old process

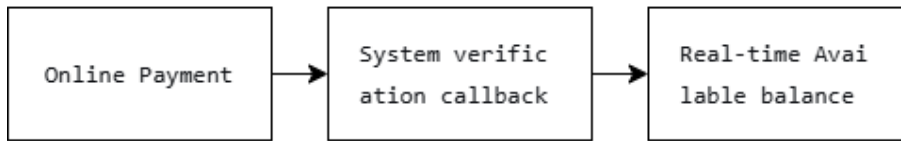


Figure 4: Diagram of new process

3.3 System Boundary and Responsibility Definition

To ensure a clear project scope, we explicitly define the boundaries of CCSS’s responsibilities:

- **System Responsibilities**
 - Manage digital information of users and campus card accounts.

- Integrate and maintain reliable connections with WeChat Pay and Alipay payment gateways.
- Process payment callbacks and execute the core ledger logic of increasing user balances.
- Record complete and immutable transaction logs for all transactions.
- Provide frontend interfaces for users to query balances and consumption history.
- **System Non-Responsibilities**
 - Handle, hold, or transfer any physical funds.
 - Operate the university’s bank corporate account.
 - Process any form of withdrawal, refund, or fund transfer-out requests.

4 User Requirement Analysis

4.1 System Requirements

The transition from the initial problem investigation to the technical specification is driven by a direct mapping of identified operational pain points to explicit system behaviors. By analyzing the structural flaws of Jinan University’s legacy process—specifically the friction caused by the offline “Interim Balance” loading step and the single-channel limitation—the proposed solution mandates a pure online, multi-channel architecture.

1. Account Management Module (AM)

- (a) **REQ-AM-1:** The system shall allow users to securely bind their physical or virtual campus cards to their mobile web account. **(Priority: 4/5)**
- (b) **REQ-AM-2:** The system shall provide a real-time user interface for teachers and students to query their current “Card Balance”. **(Priority: 5/5)**
 - i. **REQ-AM-2a:** The system shall push real-time balance updates to the frontend via WebSockets immediately following any balance-altering transaction. **(Priority: 3/5)**
 - ii. **REQ-AM-2b:** The system shall verify user identity using JWT tokens for all account-related frontend queries. **(Priority: 5/5)**

2. Recharge Service Module (RS)

- (a) **REQ-RS-1:** The system shall allow users to initiate a top-up transaction by selecting a customized or preset monetary amount within the H5 application. **(Priority: 4/5)**
 - i. **REQ-RS-1a:** The system shall generate a unique recharge order with an initial status of “pending payment” upon user initiation. **(Priority: 4/5)**
 - ii. **REQ-RS-1b:** The system shall securely invoke third-party payment gateways (WeChat Pay and Alipay) by generating and returning valid prepayment parameters to the frontend. **(Priority: 4/5)**

- (b) **REQ-RS-2:** The system shall expose public API endpoints to receive asynchronous HTTP payment callbacks from WeChat Pay and Alipay. (**Priority: 5/5**)
 - i. **REQ-RS-2a:** The system shall directly and atomically credit the successful top-up amount to the user's "Card Balance" upon valid callback confirmation, completely bypassing any "Interim Balance" or offline loading step. (**Priority: 5/5**)
 - ii. **REQ-RS-2b:** The system shall provide real-time interactive UI feedback ("Top-up successful, XX yuan has been credited") upon a successful balance update. (**Priority: 2/5**)

3. Consumption Deduction Module (CD)

- (a) **REQ-CD-1:** The system shall expose a secure deduction interface accessible by campus intranet/VPN-connected POS terminals. (**Priority: 5/5**)
 - i. **REQ-CD-1a:** The system shall verify that an account's balance is greater than or equal to the requested deduction amount before executing a transaction. (**Priority: 5/5**)
 - ii. **REQ-CD-1b:** The system shall execute deductions atomically using row-level database locks (SELECT ... FOR UPDATE) to prevent concurrent overspending. (**Priority: 5/5**)
 - iii. **REQ-CD-1c:** The system shall actively evict/delete the user's balance cache in Redis immediately following a successful database transaction commit to ensure data freshness. (**Priority: 3/5**)

4. Transaction Log Management Module (TL)

- (a) **REQ-TL-1:** The system shall automatically record an immutable transaction log record for every single recharge and consumption event. (**Priority: 4/5**)
- (b) **REQ-TL-2:** The system shall support multi-dimensional transaction log queries allowing users to filter history. (**Priority: 2/5**)
 - i. **REQ-TL-2a:** The system shall allow filtering of transaction logs by time range, transaction type, and merchant. (**Priority: 1/5**)
 - ii. **REQ-TL-2b:** The system shall limit user transaction log queries to a maximum time span of one year per request. (**Priority: 2/5**)

5. Payment Gateway Integration Module (PG)

- (a) **REQ-PG-1:** The system shall provide a unified payment gateway adapter layer to abstract and encapsulate the underlying API variations between WeChat Pay and Alipay. (**Priority: 3/5**)

6. Security & Reliability Requirements (SR)

- (a) **REQ-SR-1:** The system shall enforce rigorous runtime security checks on all inbound API traffic.

- i. **REQ-SR-1a:** The system shall validate payment callback legitimacy using cryptographic signature verification at both the API Gateway and Business Service layers. **(Priority: 5/5)**
- ii. **REQ-SR-1b:** The system shall strictly prevent horizontal privilege escalation by enforcing that the `user_id` requested in balance or log queries perfectly matches the internal ID decrypted from the caller's JWT token. **(Priority: 5/5)**
- iii. **REQ-SR-1c:** The system shall authenticate POS terminals utilizing a pre-shared secret key signature mechanism, tracking a unique `terminal_id`, a timestamp window, and a random nonce to prevent replay attacks. **(Priority: 4/5)**
- (b) **REQ-SR-2:** The system shall guarantee absolute data integrity and process idempotency across all shared services.
 - i. **REQ-SR-2a:** The system shall guarantee absolute process idempotency for all recharge callbacks by verifying the transaction state prior to executing database changes. **(Priority: 4/5)**
 - ii. **REQ-SR-2b:** The system shall employ Redis distributed locks (SETNX) on unique transaction numbers (`out_trade_no`) to ensure duplicate POS deduction requests do not duplicate ledger entries. **(Priority: 4/5)**
 - iii. **REQ-SR-2c:** The system shall log all failed recharge callbacks, deduction anomalies, and mismatched balance errors, triggering real-time alerts for manual operations auditing. **(Priority: 2/5)**

7. System Boundary Constraints (Strict Non-Requirements) (CON)

- (a) **REQ-CON-1:** The system shall not hold, manage, store, or physically transfer any monetary funds. **(Priority: 5/5)**
- (b) **REQ-CON-2:** The system shall not have operational or execution authority over the university's corporate bank account. **(Priority: 5/5)**
- (c) **REQ-CON-3:** The system shall reject any user or terminal request for cash withdrawals, refunds, or outward fund transfers. **(Priority: 5/5)**

4.2 Acceptance Tests

To verify that the system accurately implements the engineering requirements, the acceptance tests are designed using a behavioral Black-Box testing paradigm structured around clear operational boundaries.

1. Account Management Module (AM)

- (a) **ATC-AM-1-1:** Given a user with a valid campus identity, completing the binding process in the application (pass: the system successfully links the virtual card profile and displays a "bound" status).

- (b) **ATC-AM-2-1:** Given an authenticated user on the H5 dashboard, loading the main account interface (pass: the system retrieves and displays the correct current "Card Balance" from the database).

2. Recharge Service Module (RS)

- (a) **ATC-RS-1-1:** Given an authenticated user on the top-up page, selecting a monetary amount and clicking "recharge" (pass: the system generates a pending order and initiates the third-party payment interface).
- (b) **ATC-RS-2-1:** Given a pending recharge order, receiving a valid asynchronous payment callback from WeChat Pay or Alipay (pass: the system directly increments the account's "Card Balance" without creating an interim balance).

3. Consumption Deduction Module (CD)

- (a) **ATC-CD-1-1:** Given a user account with a sufficient balance, initiating a payment request from an authenticated campus POS terminal (pass: the system atomically deducts the specified amount and returns the updated balance to the terminal).

4. Transaction Log Management Module (TL)

- (a) **ATC-TL-1-1:** Given a successfully processed recharge or consumption transaction, querying the transaction ledger database (pass: the system writes a permanent, immutable record containing the precise transaction details).
- (b) **ATC-TL-2-1:** Given an authenticated user viewing their profile, applying search filters on the history page (pass: the system returns a matching list of historical transaction records).

5. Payment Gateway Integration Module (PG)

- (a) **ATC-PG-1-1:** Given a payment request directed to either WeChat Pay or Alipay, routing the transaction through the payment adapter layer (pass: the system uniformly standardizes order creation and signature verification across both platforms).

5 User Requirement Specification

5.1 Use Cases

The functional boundaries and system constraints specified in the previous sections are operationalized through a rigorous behavioral model. By translating technical specifications into concrete, goal-driven interactions, this section defines the precise operational sequences that bridge human users, hardware devices, and external cloud infrastructure.

1. Account Management Module (AM)

UC-AM-1: Bind Campus Card

Related Requirements:	REQ-AM-1
Extension Points:	None.
Actors:	Initiating: Campus Card Holder (Goal: Link their physical/virtual identity to the digital application).
Preconditions:	The user has successfully logged into the H5 application container but has no active card associated with their account profile.
Success End Condition (Pass):	The system successfully links the virtual card profile, updates the database, and displays a “bound” status on screen.
Failed End Condition (Fail):	The system rejects the binding request, logs the validation failure, and leaves the user account in an unbound state.
Flow of Events:	

1. Main Success Scenario (MSS):

1. User navigates to the card management section of the H5 application.
2. System displays a form requesting campus identity credentials (e.g., student/faculty ID number, password).
3. User enters their credentials and submits the form.
4. System validates the credentials against the school’ s central registry database.
5. System creates an atomic database mapping between the user’ s digital profile and the campus card profile.
6. System updates the UI to show a “bound” status and displays the card metadata.

2. Alternative Scenarios:

(a) Step 4a: Invalid Credentials:

1. System detects a mismatch or non-existent identity in the central registry.
2. System displays an error message: “Authentication failed. Please verify your credentials.”
3. System returns to Step 2.

UC-AM-2: Check Real-Time Balance

Related Requirements:	REQ-AM-2, REQ-AM-2b, REQ-SR-1b
Extension Points:	UC-AM-3 (Triggered automatically at the end of the flow to maintain live sync).
Actors:	Initiating: Campus Card Holder (Goal: View their accurate, current spendable card balance).
Preconditions:	The user is fully authenticated, holds a valid JWT token, and has a bound campus card.
Success End Condition (Pass):	The system securely retrieves and displays the correct current “Card Balance” from the database.
Failed End Condition (Fail):	The system blocks access, returns an error status code (e.g., 403 Forbidden), and shields the financial data.
Flow of Events:	

1. Main Success Scenario (MSS):

1. User accesses or refreshes the H5 application home dashboard.
2. Frontend sends an HTTP GET request to the balance endpoint, embedding the JWT token in the header.
3. API Gateway intercepts the request, decrypts the JWT token, and verifies that the payload user_id matches the request parameters.
4. System queries the read replica database to fetch the current balance field associated with the validated user ID.
5. System assembles a JSON payload containing the card number and current balance, sending it to the frontend.
6. System displays the exact monetary balance on the user’s dashboard screen.

2. Alternative Scenarios:

(a) Step 3a: Horizontal Privilege Escalation Detected:

1. API Gateway detects that the requested user_id parameter does not match the internal ID embedded within the validated JWT token.
2. System blocks the request, generates a security log entry, and returns a 403 Forbidden error code.

(b) Step 3b: Expired or Invalid Token:

1. API Gateway fails to decrypt or verify the cryptographic signature of the JWT token.
2. System returns a 401 Unauthorized error code and redirects the frontend to the login page.

UC-AM-3: Receive Instant Balance Update

Related Requirements:	REQ-AM-2a
Extension Points:	None.
Actors:	Initiating: CCSS Core Engine (Non-human; Goal: Push asynchronous ledger updates to client environment). Participating: Campus Card Holder (Goal: Receive immediate confirmation without reloading the app).
Preconditions:	The user has the H5 application open, and a WebSocket connection is successfully established and mapped to their user ID channel.
Success End Condition (Pass):	The application handles an incoming push event and dynamically refreshes the on-screen balance total without user intervention.
Failed End Condition (Fail):	The push message fails to deliver or parse, leaving the stale balance on screen until a manual page pull-to-refresh is executed.

Flow of Events:

1. Main Success Scenario (MSS):

1. System commits a transaction that alters the user' s balance database record (via top-up or consumption).
2. System publishes a balance-change message to the corresponding Redis message channel.
3. WebSocket service captures the Redis message and identifies the user' s active socket connection.
4. System pushes the new balance JSON payload down to the active H5 frontend client.
5. H5 application receives the event payload, parses the balance field, and dynamically updates the UI screen.

2. Alternative Scenarios:

(a) Step 3a: WebSocket Connection Dropped:

1. System attempts to locate an active connection channel but finds the socket closed due to a client network switch.
2. System drops the push message and relies on the client initiating a standard HTTP GET request (via UC-AM-2) upon reconnection.

2. Recharge Service Module (RS)

UC-RS-1: Initiate Top-Up Order

Related Requirements:	REQ-RS-1, REQ-RS-1a, REQ-RS-1b, REQ-PG-1
Extension Points:	None.
Actors:	Initiating: Campus Card Holder (Goal: Add funds to their digital card ledger via mobile payment). Participating: Third-Party Payment Platform (WeChat Pay / Alipay) (Goal: Receive unified order requests).
Preconditions:	The user is authenticated and has a bound campus card account in good standing.
Success End Condition (Pass):	The system generates a pending order and initiates the third-party payment interface by passing back valid prepayment parameters.
Failed End Condition (Fail):	The system displays an error message, declines order generation, and does not open a third-party checkout overlay.

Flow of Events:

1. Main Success Scenario (MSS):

1. User navigates to the top-up page, selects or inputs a monetary amount, and chooses a payment channel (WeChat Pay or Alipay).
2. User clicks the “Confirm Recharge” button.
3. System generates a unique internal order record in the master database with a status of “pending payment”.
4. Payment Gateway Adapter interceptor transforms the request into the specific API contract required by the chosen provider (WeChat Pay or Alipay).
5. System invokes the external payment platform’ s unified order API.
6. Third-Party Payment Platform processes the order creation, registers the callback URL, and returns secure prepayment string tokens to the CCSS backend.
7. System packages these tokens and returns them to the H5 frontend, which automatically opens the platform’ s native payment overlay.

2. Alternative Scenarios:

(a) Step 5a: External Gateway Timeout:

1. System detects that WeChat Pay or Alipay APIs fail to respond within the configured timeout window.

2. System marks the local order as “failed creation” and displays an error message: “Payment gateway unavailable. Please try again later.”

UC-RS-2: Process Payment Callback

Related Requirements:	REQ-RS-2, REQ-RS-2a, REQ-SR-1a, REQ-SR-2a
Extension Points:	UC-RS-3 (Triggered at Step 6 to push frontend alerts), UC-TL-1 (Triggered at Step 4 to record history).
Actors:	Initiating: Third-Party Payment Platform (WeChat Pay / Alipay) (Goal: Deliver authenticated transaction confirmation). Participating: Corporate Bank Account System (Goal: Serve as the destination for physical fund settlement).
Preconditions:	A user has finalized an payment overlay transaction, causing money to transfer into the university’ s physical bank account.
Success End Condition (Pass):	The system directly increments the account’ s “Card Balance” without creating an interim balance, and responds with a success acknowledgment code.
Failed End Condition (Fail):	The transaction rolls back, balances are left unchanged, and an error signal is returned to the platform to trigger a retry.
Flow of Events:	

1. Main Success Scenario (MSS):

1. Third-Party Payment Platform completes the settlement to the school’ s bank account and sends an asynchronous HTTP callback payload to the CCSS public gateway.
2. API Gateway and Recharge Service execute cryptographic signature verification using the provider’ s public keys to confirm authenticity.
3. System initiates an internal database check to ensure the order is still “pending payment” (enforcing process idempotency).
4. System opens a high-integrity database transaction: updates the local order status to “paid”, atomically increments the user’ s ledger “Card Balance” field directly, and commits the transaction.
5. System executes an active eviction of the user’ s cached Redis balance to ensure data consistency.
6. System outputs an exact, formatted acknowledgment string (XML/JSON success code) back to the payment platform actor.

2. Alternative Scenarios:

(a) Step 2a: Invalid Callback Signature:

1. System detects a signature mismatch on the incoming webhook payload, indicating a possible forged request.
2. System drops the payload, writes a high-priority warning log, and returns an HTTP error code to the platform.

(b) Step 3a: Duplicate Callback Received (Idempotency Trigger):

1. System detects that the target order status is already marked as “paid” in the database.
2. System completely bypasses database ledger operations, skips data updates, and goes directly to Step 6 to acknowledge receipt gracefully.

UC-RS-3: Notify Top-Up Success

Related Requirements:	REQ-RS-2b
Extension Points:	None.
Actors:	Initiating: System Notification Engine (Non-human). Participating: Campus Card Holder (Goal: Receive instant feedback confirming their balance change).
Preconditions:	UC-RS-2 has committed its database transaction successfully.
Success End Condition (Pass):	The system pushes an immediate visual confirmation to the H5 interface upon a verified database credit, alerting the user that their funds are ready to spend.
Failed End Condition (Fail):	The app screen stays on a loading page or home dashboard without a contextual text alert, requiring a manual balance review.

Flow of Events:

1. Main Success Scenario (MSS):

1. System captures the successful balance modification event published by the recharge engine.
2. System sends a targeted message through the application event layer to the client UI.
3. H5 application intercepts this signal and halts any active transaction processing loading indicators.

4. System triggers a visual dialog pop-up on the user' s interface displaying: "Top-up successful, XX yuan has been credited."

3. Consumption Deduction Module (CD)

UC-CD-1: Deduct Card Balance at POS

Related Requirements:	REQ-CD-1, REQ-CD-1a, REQ-CD-1b, REQ-CD-1c, REQ-SR-1c, REQ-SR-2b
Extension Points:	UC-TL-1 (Triggered upon transaction commit to append logs).
Actors:	Initiating: Campus POS Terminal (Goal: Safely request and record a real-time on-campus purchase deduction). Participating: POS Merchant Operator (Goal: Secure immediate transaction feedback at checkout lines).
Preconditions:	The physical POS hardware terminal is securely registered in the system backend and connected over the school' s intranet/VPN link.
Success End Condition (Pass):	The system atomically deducts the specified amount and returns the updated balance to the terminal.
Failed End Condition (Fail):	The transaction rolls back, no funds are deducted, and an explicit refusal response packet is returned to the terminal hardware.

Flow of Events:

1. Main Success Scenario (MSS):

1. POS Merchant Operator enters a transaction amount at the register, and the user scans their card identity token against the terminal reader.
2. Campus POS Terminal bundles the request data along with its terminal_id, a timestamp, a random nonce, and a unique out_trade_no, signing it with its pre-shared secret key.
3. API Gateway recalculates the signature using the terminal' s registered key and checks the timestamp window to block replay actions.
4. Consumption Service attempts a Redis SETNX lock operation on the inbound out_trade_no string to guarantee single-execution idempotency.
5. System opens a database transaction and requests a row-level write lock (SELECT ... FOR UPDATE) on the user' s card balance record.
6. System verifies the balance is greater than or equal to the checkout amount, executes an update (balance = balance - amount), commits the transaction, and evicts the Redis balance cache.

7. System builds a successful JSON confirmation payload containing the deduction status and remaining balance, returning it to the terminal screen.

2. Alternative Scenarios:

(a) Step 4a: Duplicate Request Detected (Idempotency Lock Failure):

1. Redis SETNX returns 0, indicating this precise terminal transaction id has already hit the processing pipeline.
2. Consumption Service queries the database log for the exact original execution outcome of that out_trade_no.
3. System returns that historical processing result directly to the terminal, bypassing ledger alterations.

(b) Step 6a: Insufficient Funds Accounted:

1. System discovers that the user's card balance is strictly less than the requested deduction transaction value.
2. System rolls back the database transaction, clears the temporary Redis SETNX lock key, and returns an error response string: "Error: Insufficient Balance".

4. Transaction Log Management Module (TL)

UC-TL-1: Log Transaction Record

Related Requirements:	REQ-TL-1
Extension Points:	None.
Actors:	Initiating: CCSS Core Engine (Non-human).
Preconditions:	A balance modification action has successfully written and committed data changes to the database ledger.
Success End Condition (Pass):	The system writes a permanent, immutable record containing the precise transaction details.
Failed End Condition (Fail):	The system falls out of operational alignment, throwing system warning exceptions to the administrative logs.
Flow of Events:	

1. Main Success Scenario (MSS):

1. Core Engine commits a transaction affecting a user's digital account ledger balance.
2. Transaction service receives a structured event block including the transaction identifier, source profile, transaction type, volume, merchant ID, and timestamp.
3. System issues an insert statement to place an immutable record entry directly into the transaction ledger master database tables.

4. Database logs the action and stores the block safely with write-once permissions rules.

UC-TL-2: Query Transaction History

Related Requirements:	REQ-TL-2, REQ-TL-2a, REQ-TL-2b, REQ-SR-1b
Extension Points:	None.
Actors:	Initiating: Campus Card Holder (Goal: Review a categorized, clear breakdown of past expenditures).
Preconditions:	User is authenticated via JWT token and querying an identity matching their log token.
Success End Condition (Pass):	The system returns a matching list of historical transaction records.
Failed End Condition (Fail):	The system blocks access, returns an error status code, and leaves the screen clear of sensitive financial records.
Flow of Events:	

1. Main Success Scenario (MSS):

1. User navigates to the transaction log interface and applies filters (e.g., specific merchant, transaction type, time range).
2. H5 app submits a query request containing the parameters to the gateway layer.
3. API Gateway decrypts the JWT token, confirms parameter identity alignment, and verifies the requested time span is under one year.
4. Transaction Service queries the read replica database to retrieve matching records from the transaction ledger.
5. System transforms the database rows into a clear, structured JSON array and returns it to the client.
6. H5 application renders the history records chronologically for the user.

2. Alternative Scenarios:

(a) Step 3a: Query Time Range Window Exceeded:

1. System checks validation criteria and notices the user requested a data span stretching back over a year.
2. System rejects database execution and presents an app error alert: "Queries are limited to a maximum range of one year."

UC-TL-3: Audit Failed Transactions

Related Requirements:	REQ-SR-2c
Extension Points:	None.
Actors:	Initiating: System Auditing Service (Non-human).
Preconditions:	An unhandled error or exception breaks a running process chain during recharge callbacks or deduction transactions.
Success End Condition (Pass):	The system monitors and flags anomalous execution steps or mismatched ledger entries, generating real-time operational alerts for manual technical auditing.
Failed End Condition (Fail):	System failures occur silently, causing data drifts or missed operational issues without alerting technical support staff.
Flow of Events:	

1. Main Success Scenario (MSS):

1. An unhandled exception or transaction failure occurs during a recharge or deduction operation.
2. System intercepts the failure, logs the detailed stack trace and payload state to the system log, and rolls back any pending database operations.
3. Auditing service matches the exception pattern against known operational anomaly threshold rules.
4. System sends an immediate alert notification detailing the event out to the technical operations dashboard.

5.2 Acceptance Tests

We now refactor the acceptance tests to let it correspond to use cases instead of primitive requirements.

1. Account Management Module (AM)

ATC-AM-1-1: Bind Campus Card Test

Related Requirements:	UC-AM-1: Bind Campus Card
Input Data Description:	An authenticated mobile web container context containing an unbound profile, combined with the manual submission of unmapped, structurally valid campus authentication credentials (student/faculty identity number and associated password string). For the exception branch, the action includes submitting structurally valid credentials where the password does not match the identity mapping.
Pass Criteria:	The system successfully maps the external single-page container account to the verified internal database card profile, writing a success state and returning a "bound" response.
Fail Criteria:	The system rejects the transaction, outputs a specific validation error, and ensures the user account remains completely unbound.
Test Procedure:	
1. Main Execution Sequence:	1. Authenticate the client environment container and navigate to the card binding interface. 2. Input the structurally valid campus identity number and matching identity password into the form fields. 3. Trigger the form submission action. 4. (Exception Branch) Clear the active application session, re-initiate the card binding view, and submit valid identity numbers accompanied by an incorrect password string.
Expected Results:	1. Main Success Scenario: Over the standard successful lifecycle, the application initializes a clean capture form, securely hashes the inbound request transmission, and checks it against the central school registry. Upon verification, the backend executes an atomic database commit to link the user and card tables. The single-page frontend catches the resolved response state to dynamically transform the interface, displaying a successful "bound" status overlay along with the correct card metadata. 2. Exception Branch Scenario: Over the exception branch execution, the backend gateway halts processing at the credential validation layer, blocks all secondary database state insertions, and surfaces a localized screen alert informing the user that authentication failed and credentials must be verified.

ATC-AM-2-1: Check Real-Time Balance Test

Related Requirements:	UC-AM-2: Check Real-Time Balance
Input Data Description:	An HTTP header packet containing a cryptographically signed, unexpired JSON Web Token (JWT), accompanied by an implicit or explicit user profile identifier request. For the security attack branch, the user action consists of manually tampering with the request parameters to query a different user's unique profile ID while retaining the original authorization token.
Pass Criteria:	The system matches the header token context, safely reads the database read-replica ledger, and displays the exact monetary value on the dashboard screen.
Fail Criteria:	The system detects an identity mismatch, drops the request pipeline, blocks data delivery, and returns a 403 Forbidden error payload.

Test Procedure:

1. Main Execution Sequence:

1. Initialize the single-page application home dashboard view under an active user context containing a valid JWT.
2. Trigger a dashboard refresh action to send an automated read request to the balance endpoint.
3. (Security Attack Branch) Intercept the outgoing API request packet, replace the requested user profile parameter with an unauthorized profile identifier, and allow the packet to transmit.

Expected Results:

1. **Main Success Scenario:** In the normal workflow, the API gateway intercepts the request, decrypts the token signature, and verifies that the internal caller ID matches the path parameter. The system queries the low-latency database read-replica tables to extract the account balance field, compiles a secure JSON payload, and forces the frontend view to display the precise spendable balance text on the home screen.
2. **Security Attack Scenario:** In the security attack scenario, the backend gateway boundary identifies a parameter mismatch against the decrypted token contents, instantly dropping the pipeline execution. No balance information from the database layer is exposed, and the system issues a strict 403 Forbidden response code back to the client container.

2. Recharge Service Module (RS)

ATC-RS-1-1: Initiate Top-Up Order Test

Related Requirements:	UC-RS-1: Initiate Top-Up Order
Input Data Description:	The user action of opening the recharge module, typing a positive numeric value within permitted threshold limits into the amount input field, choosing an active third-party channel provider (WeChat Pay or Alipay), and clicking the checkout confirmation button. For the exception branch, the environment inputs an artificial gateway network disconnection script to trigger a platform timeout during communication.
Pass Criteria:	The system successfully inserts a fresh ledger order row marked as "pending payment" and returns signed, valid prepay parameters capable of launching the native platform checkout overlay.
Fail Criteria:	The system throws an explicit connection error, aborts local order state creation, and does not invoke external gateway parameters.

Test Procedure:

1. Main Execution Sequence:

1. Access the wallet top-up interface inside the H5 browser wrapper, pick an active channel selector, and input a positive monetary value.
2. Submit the top-up transaction request by clicking the confirmation button.
3. (Exception Branch) Set the gateway simulation proxy to automatically drop or freeze outbound traffic destined for external payment APIs, then repeat the initiation sequence.

Expected Results:

1. **Main Success Scenario:** During a successful execution lifecycle, the frontend validates the amount format and transmits the payload. The system inserts an order record marked "pending payment" into the master database, and the gateway adapter normalizes the schema contract before calling the external payment provider's API. The external platform processes the request, registers the callback path, and returns cryptographic prepay tokens, which the CCSS backend forwards to the H5 client to successfully trigger the native platform checkout overlay.

2. **Exception Branch Scenario:** During the timeout exception branch, the backend adapter catches the socket connection threshold breach, aborts any local database order state finalization, and flags the transaction log with a creation anomaly. The frontend intercepts this failure response to present a clear, consumer-facing error message stating that the payment gateway is currently unavailable.

ATC-RS-2-1: Process Payment Callback Test

Related Requirements: UC-RS-2: Process Payment Callback

Input Data Description: An asynchronous incoming HTTP POST callback payload delivered to the public gateway URL containing transaction data blocks, a valid internal reference number, and a verifiable cryptographic platform signature. For the attack scenario, the input data consists of an altered payment status notification payload where the cryptographic signature has been intentionally modified or corrupted.

Pass Criteria: The backend securely validates the signature, performs an atomic, direct database balance adjustment bypassing intermediate storage, clears the Redis cache, and returns a success verification code to the sender.

Fail Criteria: The transaction rolls back, balances are left unchanged, and an error signal is returned to the platform to trigger a retry.

Test Procedure:

1. Main Execution Sequence:

1. Issue a mock asynchronous HTTP POST payment notification payload directly to the public-facing gateway endpoint, embedding a valid internal order tracking reference and an authentic channel signature.
2. (Exception Branch) Issue an identical notification payload containing a valid tracking reference but manipulate the cryptographic signature parameter string to introduce a verification failure.

Expected Results:

1. **Main Success Scenario:** In the main success scenario, the gateway intercepts the webhook and validates its authenticity using the payment provider's public key. The recharge service verifies the order's status as "pending payment," opens an exclusive database transaction lock, updates the order status to "paid," and credits the account's usable "Card Balance"

field directly, completely bypassing any "Interim Balance" or offline loading step. It then clears the Redis cache and returns an explicit success confirmation string to the external provider.

2. **Exception Branch Scenario:** In the corrupted signature scenario, the validation layer flags the structural forgery attempt and terminates execution before hitting the ledger tables. The database remains completely unchanged, and the gateway returns an execution fault code back to the sender to prevent improper balance credits.

3. Consumption Deduction Module (CD)

ATC-CD-1-1: Deduct Card Balance at POS Test

Related Requirements:	UC-CD-1: Deduct Card Balance at POS
Input Data Description:	A hardware interaction event consisting of a card token scan at an intranet-connected POS terminal, combined with an input purchase amount and a terminal-generated signature package containing a unique transaction identifier, timestamp, and hardware key token. For the exception scenario, the input involves a card token scan linked to an account whose balance is lower than the input purchase amount.
Pass Criteria:	The system processes the signed intranet package, implements an exclusive row lock on the data table, runs safety margin checks, deducts the price, flushes Redis, and sends an updated balance statement to the terminal.
Fail Criteria:	The system rolls back ledger updates, releases temporary locking elements, and transfers a clear transaction failure error string to the POS terminal hardware.
Test Procedure:	
1. Main Execution Sequence:	
	1. Trigger a signed deduction transaction payload from an intranet-connected physical campus POS terminal component where the associated account balance exceeds the transaction amount. 2. (Exception Branch) Trigger an identical deduction transaction sequence from the terminal component, but target an account whose balance is lower than the requested purchase amount.
Expected Results:	

1. **Main Success Scenario:** In the standard success scenario, the API gateway validates the terminal signature and checks the timestamp window to block replay attacks. The consumption service secures a single-execution idempotency lock in Redis using the transaction identifier, executes a row-level write lock (SELECT ... FOR UPDATE) on the database ledger, validates the funds margin, decrements the balance, and evicts the corresponding Redis cache. The system commits the transaction and transmits a JSON payload to the hardware terminal confirming the deduction and displaying the remaining balance.
2. **Exception Branch Scenario:** In the insufficient funds exception scenario, the system halts execution immediately after evaluating the balance margin condition. The database transaction is rolled back, the temporary Redis idempotency lock is cleared, and an explicit refusal string ("Error: Insufficient Balance") is delivered to the hardware terminal screen.

6 Top UI Design

The structural requirements and behavioral workflows modeled in the previous sections are translated into a user-centric interface layer through an intuitive, minimal-click H5 application blueprint. Designed to operate seamlessly inside mobile web containers (such as WeChat and Alipay built-in browsers), the user interface architecture directly mirrors the decoupled tier boundaries, optimizing for high-frequency user actions like balance queries and instant top-ups. To eliminate the historical operational friction identified in the business logic investigation—namely the confusing intermediate storage states and physical loading bottlenecks—the screen layouts are optimized for immediate financial transparency and instant, real-time interface feedback. Each screen is structured into modular visual components that align precisely with backend validation parameters, ensuring that security actions (such as JWT-driven identity mapping) and performance frameworks (such as reactive WebSocket balance updates) function smoothly behind a responsive, modern interface.

7 System Specification

7.1 System Responsibility Boundaries

- **Responsible for:**
 - Managing users and campus card digital accounts
 - Integrating with WeChat Pay / Alipay payment gateways
 - Processing payment callbacks and directly increasing account balances
 - Recording immutable transaction logs
 - Providing interfaces for balance and transaction inquiry
- **Not Responsible for:**
 - Does not process, hold, or transfer any physical funds

- Does not operate the school's corporate bank account
- Does not support cash withdrawals, refunds, or outward fund transfers

7.2 Functional Modules

- **Account Management:** Bind campus cards, query real-time balance, manage user identity
- **Recharge Service:** Generate recharge orders, invoke third-party payments, process asynchronous callbacks, atomically update balance
- **Consumption Deduction:** Receive POS terminal deduction requests, verify balance, deduct, and return new balance
- **Transaction Log Management:** Record an immutable log for every recharge/consumption transaction; support multi-dimensional query and reconciliation
- **Payment Gateway Integration:** Uniformly encapsulate WeChat Pay and Alipay APIs for unified order creation, callback signature verification, and order query

7.3 System Architecture Design

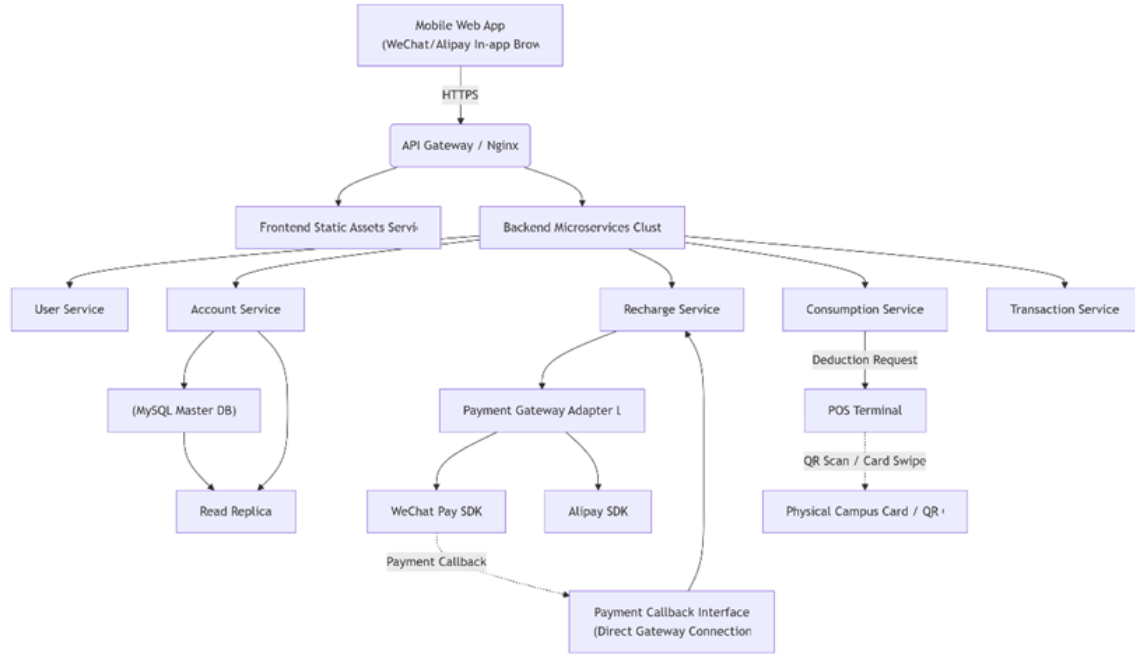


Figure 5: System Architecture Diagram

As shown in figure 5, CCSS adopts a B/S architecture as a mobile web application, with separated front-end and back-end communicating via RESTful APIs. The overall architecture is divided into five layers: Front-end Presentation Layer, Access Gateway Layer, Business Service Layer, Infrastructure Layer, and External System Layer. The system integrates with WeChat Pay / Alipay via asynchronous callbacks and interacts with offline POS terminals through a dedicated deduction interface.

Detailed description of each component is as follows:

1. **Mobile Web Frontend:** An H5 application running within the built-in browsers of WeChat or Alipay. It calls backend APIs to initiate recharges, query balances and transaction logs, and receives real-time balance updates via WebSocket.
2. **POS Terminal:** Devices in canteens, supermarkets, etc., that call the consumption deduction interface to complete balance deductions and return transaction results.
3. **API Gateway (Nginx):** A single entry point handling HTTPS termination, rate limiting, and routing. Callback interfaces are forwarded by the gateway to the Recharge Service.
4. **User Service:** Handles campus card binding, identity authentication, and user information management.
5. **Account Service:** Maintains digital campus card account balances and provides balance queries (reading from the read replica database).
6. **Recharge Service:** Generates recharge orders, calls the Payment Gateway Adapter; receives and verifies payment callbacks, performs atomic balance increases, and records recharge transaction logs.
7. **Consumption Service:** Receives POS deduction requests, verifies the balance and deducts the amount, and records consumption transaction logs.
8. **Transaction Service:** Provides transaction log query interfaces, supporting filtering by time, type, merchant, etc.
9. **Payment Gateway Adapter:** Uniformly encapsulates the unified order and order query interfaces of WeChat Pay and Alipay, shielding the underlying differences.
10. **MySQL Master Database:** Stores core data such as users, account balances, orders, and transaction logs, with strong consistency writes.
11. **Read Replica:** Handles read requests like balance and transaction log queries, reducing pressure on the master database.
12. **Redis Cache:** Caches hotspot account balances, idempotency tokens, and distributed locks; supports the message channel for WebSocket pushes.
13. **Corporate Bank Account:** Handles fund flow only. Recharge funds go directly from the user to the school's bank account; the CCSS system has zero contact with the physical funds.

7.4 Core Interface Operational Flows

1. Recharge Interface

- (a) **Create Recharge Order:** The system creates an order with a “pending payment” status, calls the payment gateway to generate prepayment parameters, and returns them to the frontend.
- (b) **Asynchronous Payment Callback**
 - i. Upon receiving the request (an XML/JSON message returned by the payment platform containing the order number, transaction serial number, payment status, etc.), signature verification is performed by both the gateway layer and the business layer.
 - ii. Query the order status to ensure idempotency. If the order is already processed, directly return a success response.
 - iii. Open a database transaction: Update the order status to “paid” → Increase the account balance by the corresponding amount → Write a recharge transaction log.
 - iv. Publish a balance change message via Redis, which is then pushed to the corresponding user’s frontend through WebSocket.
 - v. Return a “success” acknowledgement message to the payment platform.

2. Balance Inquiry Interface

- (a) **User Request:** The frontend page calls `GET /api/account/balance?user_id=xxxxxx`, carrying the JWT Token obtained after login in the Header.
- (b) **Gateway Security Check:** The gateway decrypts and verifies the JWT, extracting the logged-in user ID. The `user_id` in the request parameter must be absolutely identical to the user ID in the JWT; otherwise, a 403 Forbidden is returned directly to prevent querying others’ balances. After successful verification, the legitimate user ID is written into the `X-Auth-User-ID` header and forwarded to the Account Service.
- (c) **Result Assembly:** The service layer assembles a JSON response containing the user ID, physical card number, and current balance, which is returned to the frontend via the gateway.

3. Consumption Deduction Interface

- (a) **Terminal Authentication:** When making a request, the POS terminal must generate a signature based on a pre-shared secret key and send it along with `terminal_id`, a timestamp, and a random nonce. The gateway layer retrieves the corresponding key from the `terminal_auth` table based on the `terminal_id`, recalculates the signature, and compares them. If the signature is invalid or the timestamp is outside the allowed window (anti-replay), the request is rejected. This layer ensures that only legitimate POS terminals can initiate deductions.
- (b) **Idempotency Handling:** After receiving the request, the Consumption Service first uses Redis’s `SETNX` command to set a short-term lock key for the `out_trade_no` (the unique transaction serial number generated by the POS terminal). This atomically determines whether the request has already been processed.

- i. **If SETNX returns 0 (key already exists):** This indicates a duplicate request. The service directly queries the database for the previous processing result of this `out_trade_no` and returns it as-is to the POS terminal, ensuring the same deduction request is only processed once regardless of how many times it's sent.
 - ii. **If SETNX returns 1:** The normal deduction flow proceeds. After the flow completes, this key is retained for a reasonable period (the TTL can be extended after the transaction commits) to prevent concurrent duplicate processing in extreme edge cases.
- (c) **Atomic Deduction:** Within a transaction, the account balance and version number are read using a row lock (`SELECT ... FOR UPDATE`) to prevent overspending due to concurrent deductions.
 - i. **Check if the balance is greater than or equal to the deduction amount:**
 - **If Insufficient:** Immediately roll back the transaction, delete the Redis idempotency key (allowing a retry when the balance is sufficient later), and return an error with the current balance.
 - **If Sufficient:** Execute `UPDATE balance = balance - amount` using an optimistic lock (version check) and simultaneously insert a transaction log record (`type='consume'`, amount is a negative value, post-deduction balance is recorded). After committing the transaction, actively delete the user's balance cache to ensure the next query gets the latest balance.
- (d) **Result Return:** The service layer assembles a JSON response and returns the deduction result.

4. Transaction Log Inquiry Interface

- (a) The user enters filter criteria (optional type, time range) on the H5 page and pages through results. The frontend constructs a GET request that automatically carries the JWT Token.
- (b) The API Gateway intercepts the request, validates the JWT, and extracts the logged-in user ID. The `user_id` in the request parameter must match the user ID in the JWT exactly; otherwise, a 403 is returned. This step completely prevents horizontal privilege escalation where User A queries User B's transaction logs. After successful verification, the gateway writes the user ID into the `X-Auth-User-ID` request header and forwards it to the Transaction Service.
- (c) The Transaction Service receives the request and performs the following checks: Validates the type parameter; ensures the time format is correct, `start_time ≤ end_time`, and the time span is generally limited to no more than one year.
- (d) The Transaction Service maps the raw database fields into the JSON format agreed upon by the interface and returns the result.

7.5 Deployment and Interaction Highlights

- **Frontend:** A single-page H5 application deployed on a CDN or an Nginx static directory; communicates with backend APIs via AJAX and listens to the personal balance channel using

WebSocket.

- **Backend Services:** A microservice cluster based on Spring Boot (or Node.js, etc.). Modules for user, account, recharge, and consumption are deployed independently and decoupled via internal REST or message queues.
- **Payment Callbacks:** Callback URLs must be configured in the WeChat Pay / Alipay back offices as the gateway's public address. Callbacks first enter the gateway layer for signature verification and are then forwarded to the Recharge Service, ensuring security.
- **Consumption Terminals:** POS terminals access the consumption deduction interface via the campus intranet or a VPN. Terminals must have pre-installed signing keys to prevent forged requests.
- **Operations Monitoring:** Log all recharge callback and deduction failures, and set up alerts for anomalous balance changes to guarantee strict reconciliation between fund flows and information flows.